Week 1 Lesson 4 — Preprocessing data with Scikit

26th October 2026

1 8.3. Preprocessing data

Taken from scikit-learn

1.1 8.3.1. Standardization, or mean removal and variance scaling

The preprocessing module provides the StandardScaler utility class, which is a quick and easy way to perform the following operation on an array-like dataset:

[165]:
```
from sklearn import preprocessing
import numpy as np
X_train = np.array([[ 1., -1.,  2.],
                    [ 2.,  0.,  0.],
                    [ 0.,  1., -1.]])
scaler = preprocessing.StandardScaler().fit(X_train)
display(scaler)

display(scaler.mean_)

display(scaler.scale_)

X_scaled = scaler.transform(X_train)
display(X_scaled)
```

```
StandardScaler()
```

```
array([1.        , 0.        , 0.33333333])
```

```
array([0.81649658, 0.81649658, 1.24721913])
```

```
array([[ 0.        , -1.22474487,  1.33630621],
       [ 1.22474487,  0.        , -0.26726124],
       [-1.22474487,  1.22474487, -1.06904497]])
```

Scaled data has zero mean and unit variance:

[166]:
```
display(X_scaled.mean(axis=0))

display(X_scaled.std(axis=0))
```

```
array([0., 0., 0.])
```

```
array([1., 1., 1.])
```

StandardScaler works column by column.

- Columns represent features, such as temperature or cost.
- Rows represent observations, such as months.

Each column uses its own mean and standard deviation for scaling.

1.1.1 Mean and variance after standard scaling

`StandardScaler` scales each column independently using:

$$z = \frac{x - \mu}{\sigma}$$

where: - x is the original value. - μ is the original column mean. - σ is the original column standard deviation.

1.1.2 Example: first column

The original values are:

$$[1,\ 2,\ 0]$$

1. Calculate the mean:

$$\mu = \frac{1 + 2 + 0}{3} = 1$$

2. Calculate the variance:

Variance is the average squared distance from the mean:

$$\sigma^2 = \frac{(1-1)^2 + (2-1)^2 + (0-1)^2}{3} = \frac{2}{3}$$

`StandardScaler` divides by the number of observations (n), rather than $n - 1$.

3. Calculate the standard deviation:

$$\sigma = \sqrt{\frac{2}{3}} \approx 0.8165$$

4. Scale each value:

$$\left[\frac{1-1}{0.8165},\ \frac{2-1}{0.8165},\ \frac{0-1}{0.8165}\right] \approx [0,\ 1.2247,\ -1.2247]$$

1.1.3 Verify the scaled mean

$$\mu_z = \frac{0 + 1.2247 - 1.2247}{3} = 0$$

1.1.4 Verify the scaled variance

Using the unrounded scaled values:

$$\sigma_z^2 = \frac{(0-0)^2 + (\sqrt{3/2}-0)^2 + (-\sqrt{3/2}-0)^2}{3} = \frac{0 + 1.5 + 1.5}{3} = 1$$

Subtracting the mean centres the column at zero. Dividing by the standard deviation gives it unit variance.

These properties apply to each nonconstant column of the data used to fit the scaler. Rounded displayed values may give results slightly different from 0 and 1.

This class implements the Transformer API to compute the mean and standard deviation on a training set so as to be able to later re-apply the same transformation on the testing set. This class is hence suitable for use in the early steps of a Pipeline:

[167]:
```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_classification(random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
pipe = make_pipeline(StandardScaler(), LogisticRegression())
pipe.fit(X_train, y_train)  # apply scaling on training data

display(pipe.score(X_test, y_test))  # apply scaling on testing data, without␣
↪leaking training data.
```

```
0.96
```

1.2 8.3.1.1. Scaling features to a range

An alternative standardization is scaling features to lie between a given minimum and maximum value, often between zero and one, or so that the maximum absolute value of each feature is scaled to unit size. This can be achieved using MinMaxScaler or MaxAbsScaler, respectively.

The motivation to use this scaling includes robustness to very small standard deviations of features and preserving zero entries in sparse data.

Here is an example to scale a toy data matrix to the [0, 1] range:

[168]:
```
X_train = np.array([[ 1., -1.,  2.],
                    [ 2.,  0.,  0.],
                    [ 0.,  1., -1.]])
```

```
min_max_scaler = preprocessing.MinMaxScaler()
X_train_minmax = min_max_scaler.fit_transform(X_train)
display(X_train_minmax)
```

```
array([[0.5       , 0.        , 1.        ],
       [1.        , 0.5       , 0.33333333],
       [0.        , 1.        , 0.        ]])
```

The same instance of the transformer can then be applied to some new test data unseen during the fit call: the same scaling and shifting operations will be applied to be consistent with the transformation performed on the train data:

[169]:
```
X_test = np.array([[-3., -1.,  4.]])
X_test_minmax = min_max_scaler.transform(X_test)
display(X_test_minmax)
```

```
array([[-1.5       ,  0.        ,  1.66666667]])
```

It is possible to introspect the scaler attributes to find about the exact nature of the transformation learned on the training data:

[170]:
```
display(min_max_scaler.scale_)

display(min_max_scaler.min_)
```

```
array([0.5       , 0.5       , 0.33333333])
```

```
array([0.        , 0.5       , 0.33333333])
```

If MinMaxScaler is given an explicit `feature_range=(min, max)` the full formula is:

[171]:
```
range_min = 0
range_max = 1

X_std = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
X_scaled = X_std * (range_max - range_min) + range_min

display(X_scaled)
```

```
array([[0.12381866, 0.52439337, 0.4864274 , …, 0.49285474, 0.67906612,
        0.21207425],
       [0.84375859, 0.64635665, 0.5435557 , …, 0.82812533, 0.64916728,
        0.42922402],
       [0.55742129, 0.70255081, 0.81972143, …, 0.79152641, 0.42087826,
        0.34748043],
       …,
       [0.51666315, 0.20845695, 0.60900112, …, 0.49782028, 0.43780975,
        0.39628525],
       [0.73933131, 0.7513872 , 0.46276529, …, 0.49859413, 0.28347444,
        0.24459206],
```

```
       [0.52328794, 0.77176729, 0.53917657, …, 0.75291143, 0.58023005,
        0.34009484]])
```

MaxAbsScaler works in a very similar fashion, but scales in a way that the training data lies within the range [-1, 1] by dividing through the largest maximum value in each feature. It is meant for data that is already centered at zero or sparse data.

Here is how to use the toy data from the previous example with this scaler:

[172]:
```
X_train = np.array([[ 1., -1.,  2.],
                    [ 2.,  0.,  0.],
                    [ 0.,  1., -1.]])
max_abs_scaler = preprocessing.MaxAbsScaler()
X_train_maxabs = max_abs_scaler.fit_transform(X_train)
display(X_train_maxabs)
X_test = np.array([[ -3., -1.,  4.]])
X_test_maxabs = max_abs_scaler.transform(X_test)
display(X_test_maxabs)
display(max_abs_scaler.scale_)
```

```
array([[ 0.5, -1. ,  1. ],
       [ 1. ,  0. ,  0. ],
       [ 0. ,  1. , -0.5]])
```

```
array([[-1.5, -1. ,  2. ]])
```

```
array([2., 1., 2.])
```

1.3 8.3.2.1. Mapping to a Uniform distribution

'QuantileTransformer provides a non-parametric transformation to map the data to a uniform distribution with values between 0 and 1:

[173]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
quantile_transformer = preprocessing.
 ↪QuantileTransformer(random_state=0,n_quantiles=112)
X_train_trans = quantile_transformer.fit_transform(X_train)
X_test_trans = quantile_transformer.transform(X_test)
#display(X_train_trans)
#display(X_test_trans)
display(np.percentile(X_train[:, 0], [0, 25, 50, 75, 100]))
```

```
array([4.3, 5.1, 5.8, 6.5, 7.9])
```

This feature corresponds to the sepal length in cm. Once the quantile transformation is applied, those landmarks approach closely the percentiles previously defined:

[174]:
```
np.percentile(X_train_trans[:, 0], [0, 25, 50, 75, 100]).round(2)
```

[174]:
```
array([0.  , 0.24, 0.51, 0.74, 1.  ])
```

This can be confirmed on an independent testing set with similar remarks:

[175]:
```
display(np.percentile(X_test[:, 0], [0, 25, 50, 75, 100]).round(3))
display(np.percentile(X_test_trans[:, 0], [0, 25, 50, 75, 100]).round(2))
```

```
array([4.4  , 5.125, 5.75 , 6.175, 7.3  ])
```

```
array([0.01, 0.25, 0.48, 0.6 , 0.94])
```

1.4 8.3.2.2. Mapping to a Gaussian distribution

[176]:
```
quantile_transformer = preprocessing.QuantileTransformer(
    output_distribution='normal', random_state=0)
X_trans = quantile_transformer.fit_transform(X)
display(quantile_transformer.quantiles_)
```

```
/Users/alvaro/PycharmProjects/IATD-Machine-Learning-
Algorithms/.venv/lib/python3.12/site-
packages/sklearn/preprocessing/_data.py:2785: UserWarning: n_quantiles (1000) is
greater than the total number of samples (150). n_quantiles is set to n_samples.
  warnings.warn(
```

```
array([[4.3, 2. , 1. , 0.1],
       [4.4, 2.2, 1.1, 0.1],
       [4.4, 2.2, 1.2, 0.1],
       [4.4, 2.2, 1.2, 0.1],
       [4.5, 2.3, 1.3, 0.1],
       [4.6, 2.3, 1.3, 0.2],
       [4.6, 2.3, 1.3, 0.2],
       [4.6, 2.3, 1.3, 0.2],
       [4.6, 2.4, 1.3, 0.2],
       [4.7, 2.4, 1.3, 0.2],
       [4.7, 2.4, 1.3, 0.2],
       [4.8, 2.5, 1.4, 0.2],
       [4.8, 2.5, 1.4, 0.2],
       [4.8, 2.5, 1.4, 0.2],
       [4.8, 2.5, 1.4, 0.2],
       [4.8, 2.5, 1.4, 0.2],
       [4.9, 2.5, 1.4, 0.2],
       [4.9, 2.5, 1.4, 0.2],
       [4.9, 2.5, 1.4, 0.2],
       [4.9, 2.6, 1.4, 0.2],
       [4.9, 2.6, 1.4, 0.2],
       [4.9, 2.6, 1.4, 0.2],
       [5. , 2.6, 1.4, 0.2],
       [5. , 2.6, 1.4, 0.2],
       [5. , 2.7, 1.5, 0.2],
       [5. , 2.7, 1.5, 0.2],
```

```
[5. , 2.7, 1.5, 0.2],
[5. , 2.7, 1.5, 0.2],
[5. , 2.7, 1.5, 0.2],
[5. , 2.7, 1.5, 0.2],
[5. , 2.7, 1.5, 0.2],
[5. , 2.7, 1.5, 0.2],
[5.1, 2.7, 1.5, 0.2],
[5.1, 2.8, 1.5, 0.2],
[5.1, 2.8, 1.5, 0.3],
[5.1, 2.8, 1.5, 0.3],
[5.1, 2.8, 1.5, 0.3],
[5.1, 2.8, 1.6, 0.3],
[5.1, 2.8, 1.6, 0.3],
[5.1, 2.8, 1.6, 0.3],
[5.1, 2.8, 1.6, 0.3],
[5.2, 2.8, 1.6, 0.4],
[5.2, 2.8, 1.6, 0.4],
[5.2, 2.8, 1.6, 0.4],
[5.2, 2.8, 1.7, 0.4],
[5.3, 2.8, 1.7, 0.4],
[5.4, 2.8, 1.7, 0.4],
[5.4, 2.9, 1.7, 0.4],
[5.4, 2.9, 1.9, 0.5],
[5.4, 2.9, 1.9, 0.6],
[5.4, 2.9, 3. , 1. ],
[5.4, 2.9, 3.3, 1. ],
[5.5, 2.9, 3.3, 1. ],
[5.5, 2.9, 3.5, 1. ],
[5.5, 2.9, 3.5, 1. ],
[5.5, 2.9, 3.6, 1. ],
[5.5, 2.9, 3.7, 1. ],
[5.5, 3. , 3.8, 1.1],
[5.5, 3. , 3.9, 1.1],
[5.6, 3. , 3.9, 1.1],
[5.6, 3. , 3.9, 1.2],
[5.6, 3. , 4. , 1.2],
[5.6, 3. , 4. , 1.2],
[5.6, 3. , 4. , 1.2],
[5.6, 3. , 4. , 1.2],
[5.7, 3. , 4. , 1.3],
[5.7, 3. , 4.1, 1.3],
[5.7, 3. , 4.1, 1.3],
[5.7, 3. , 4.1, 1.3],
[5.7, 3. , 4.2, 1.3],
[5.7, 3. , 4.2, 1.3],
[5.7, 3. , 4.2, 1.3],
[5.7, 3. , 4.2, 1.3],
[5.8, 3. , 4.3, 1.3],
```

```
[5.8, 3. , 4.3, 1.3],
[5.8, 3. , 4.4, 1.3],
[5.8, 3. , 4.4, 1.3],
[5.8, 3. , 4.4, 1.3],
[5.8, 3. , 4.4, 1.4],
[5.8, 3. , 4.5, 1.4],
[5.9, 3. , 4.5, 1.4],
[5.9, 3. , 4.5, 1.4],
[5.9, 3. , 4.5, 1.4],
[6. , 3.1, 4.5, 1.4],
[6. , 3.1, 4.5, 1.4],
[6. , 3.1, 4.5, 1.4],
[6. , 3.1, 4.5, 1.5],
[6. , 3.1, 4.6, 1.5],
[6. , 3.1, 4.6, 1.5],
[6.1, 3.1, 4.6, 1.5],
[6.1, 3.1, 4.7, 1.5],
[6.1, 3.1, 4.7, 1.5],
[6.1, 3.1, 4.7, 1.5],
[6.1, 3.1, 4.7, 1.5],
[6.1, 3.2, 4.7, 1.5],
[6.2, 3.2, 4.8, 1.5],
[6.2, 3.2, 4.8, 1.5],
[6.2, 3.2, 4.8, 1.5],
[6.2, 3.2, 4.8, 1.6],
[6.3, 3.2, 4.9, 1.6],
[6.3, 3.2, 4.9, 1.6],
[6.3, 3.2, 4.9, 1.6],
[6.3, 3.2, 4.9, 1.7],
[6.3, 3.2, 4.9, 1.7],
[6.3, 3.2, 5. , 1.8],
[6.3, 3.2, 5. , 1.8],
[6.3, 3.2, 5. , 1.8],
[6.3, 3.3, 5. , 1.8],
[6.4, 3.3, 5.1, 1.8],
[6.4, 3.3, 5.1, 1.8],
[6.4, 3.3, 5.1, 1.8],
[6.4, 3.3, 5.1, 1.8],
[6.4, 3.3, 5.1, 1.8],
[6.4, 3.4, 5.1, 1.8],
[6.4, 3.4, 5.1, 1.8],
[6.5, 3.4, 5.1, 1.8],
[6.5, 3.4, 5.2, 1.9],
[6.5, 3.4, 5.2, 1.9],
[6.5, 3.4, 5.3, 1.9],
[6.5, 3.4, 5.3, 1.9],
[6.6, 3.4, 5.4, 1.9],
[6.6, 3.4, 5.4, 2. ],
```

```
       [6.7, 3.4, 5.5, 2. ],
       [6.7, 3.4, 5.5, 2. ],
       [6.7, 3.4, 5.5, 2. ],
       [6.7, 3.5, 5.6, 2. ],
       [6.7, 3.5, 5.6, 2. ],
       [6.7, 3.5, 5.6, 2.1],
       [6.7, 3.5, 5.6, 2.1],
       [6.7, 3.5, 5.6, 2.1],
       [6.8, 3.5, 5.6, 2.1],
       [6.8, 3.6, 5.7, 2.1],
       [6.8, 3.6, 5.7, 2.1],
       [6.9, 3.6, 5.7, 2.2],
       [6.9, 3.6, 5.8, 2.2],
       [6.9, 3.7, 5.8, 2.2],
       [6.9, 3.7, 5.8, 2.3],
       [7. , 3.7, 5.9, 2.3],
       [7.1, 3.8, 5.9, 2.3],
       [7.2, 3.8, 6. , 2.3],
       [7.2, 3.8, 6. , 2.3],
       [7.2, 3.8, 6.1, 2.3],
       [7.3, 3.8, 6.1, 2.3],
       [7.4, 3.8, 6.1, 2.3],
       [7.6, 3.9, 6.3, 2.4],
       [7.7, 3.9, 6.4, 2.4],
       [7.7, 4. , 6.6, 2.4],
       [7.7, 4.1, 6.7, 2.5],
       [7.7, 4.2, 6.7, 2.5],
       [7.9, 4.4, 6.9, 2.5]])
```

1.5 8.3.3. Normalization

Normalization is the process of scaling individual samples to have unit norm. This process can be useful if you plan to use a quadratic form such as the dot-product or any other kernel to quantify the similarity of any pair of samples.

This assumption is the base of the Vector Space Model often used in text classification and clustering contexts.

The function normalize provides a quick and easy way to perform this operation on a single array-like dataset, either using the l1, l2, or max norms:

[177]:
```
X = [[ 1., -1.,  2.],
     [ 2.,  0.,  0.],
     [ 0.,  1., -1.]]
X_normalized = preprocessing.normalize(X, norm='l2')

display(X_normalized)
```

```
array([[ 0.40824829, -0.40824829,  0.81649658],
```

```
       [ 1.        ,  0.        ,  0.        ],
       [ 0.        ,  0.70710678, -0.70710678]])
```

[178]:
```
normalizer = preprocessing.Normalizer().fit(X)  # fit does nothing
display(normalizer)
```

```
Normalizer()
```

[179]:
```
display(normalizer.transform(X))

display(normalizer.transform([[-1.,  1., 0.]]))
```

```
array([[ 0.40824829, -0.40824829,  0.81649658],
       [ 1.        ,  0.        ,  0.        ],
       [ 0.        ,  0.70710678, -0.70710678]])

array([[-0.70710678,  0.70710678,  0.        ]])
```

1.6 8.3.4. Encoding categorical features

Often features are not given as continuous values but categorical. For example a person could have features [“male”, “female”], [“from Europe”, “from US”, “from Asia”], [“uses Firefox”, “uses Chrome”, “uses Safari”, “uses Internet Explorer”]. Such features can be efficiently coded as integers, for instance [“male”, “from US”, “uses Internet Explorer”] could be expressed as [0, 1, 3] while [“female”, “from Asia”, “uses Chrome”] would be [1, 2, 1].

To convert categorical features to such integer codes, we can use the OrdinalEncoder. This estimator transforms each categorical feature to one new feature of integers (0 to n_categories - 1):

[180]:
```
enc = preprocessing.OrdinalEncoder()
X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses␣
 ↪Firefox']]
display(enc.fit(X))
display(enc.transform([['female', 'from US', 'uses Safari']]))
```

```
OrdinalEncoder()

array([[0., 1., 1.]])
```

Such integer representation can, however, not be used directly with all scikit-learn estimators, as these expect continuous input, and would interpret the categories as being ordered, which is often not desired (i.e. the set of browsers was ordered arbitrarily).

By default, OrdinalEncoder will also passthrough missing values that are indicated by np.nan.

[181]:
```
enc = preprocessing.OrdinalEncoder()
X = [['male'], ['female'], [np.nan], ['female']]
enc.fit_transform(X)
```

[181]:
```
array([[ 1.],
       [ 0.],
       [nan],
```

```
       [ 0.]])
```

If you want codes assigned in your original order, specify it explicitly:

[182]:
```
from sklearn import preprocessing

X = [
    ['male', 'from US', 'uses Safari'],
    ['female', 'from Europe', 'uses Firefox']
]

enc = preprocessing.OrdinalEncoder(categories=[
    ['male', 'female'],             # Column 1
    ["from Europe", "from US", "from Asia"],      # Column 2
    ["uses Firefox", "uses Chrome", "uses Safari", "uses Internet Explorer"]  #␣
 ↪Column 3
])

enc.fit(X)

display(enc.transform([
    ['female', 'from US', 'uses Safari']
]))
# array([[1., 0., 0.]])
```

```
array([[1., 1., 2.]])
```

OrdinalEncoder provides a parameter encoded_missing_value to encode the missing values without the need to create a pipeline and using SimpleImputer.

[183]:
```
enc = preprocessing.OrdinalEncoder(encoded_missing_value=-1)
X = [['male'], ['female'], [np.nan], ['female']]
enc.fit_transform(X)
```

[183]:
```
array([[ 1.],
       [ 0.],
       [-1.],
       [ 0.]])
```

The above processing is equivalent to the following pipeline:

[184]:
```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
enc = Pipeline(steps=[
    ("encoder", preprocessing.OrdinalEncoder()),
    ("imputer", SimpleImputer(strategy="constant", fill_value=-1)),
])
enc.fit_transform(X)
```

[184]:
```
array([[ 1.],
       [ 0.],
       [-1.],
       [ 0.]])
```

Another possibility to convert categorical features to features that can be used with scikit-learn estimators is to use a one-of-K, also known as one-hot or dummy encoding. This type of encoding can be obtained with the OneHotEncoder, which transforms each categorical feature with n_categories possible values into n_categories binary features, with one of them 1, and all others 0.

Continuing the example above:

[185]:
```
enc = preprocessing.OneHotEncoder()
X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses␣
 ↪Firefox']]
enc.fit(X)
enc.transform([['female', 'from US', 'uses Safari'],
               ['male', 'from Europe', 'uses Safari']]).toarray()
```

[185]:
```
array([[1., 0., 0., 1., 0., 1.],
       [0., 1., 1., 0., 0., 1.]])
```

[186]:
```
enc.categories_
```

[186]:
```
[array(['female', 'male'], dtype=object),
 array(['from Europe', 'from US'], dtype=object),
 array(['uses Firefox', 'uses Safari'], dtype=object)]
```

[187]:
```
genders = ['female', 'male']
locations = ['from Africa', 'from Asia', 'from Europe', 'from US']
browsers = ['uses Chrome', 'uses Firefox', 'uses IE', 'uses Safari']
enc = preprocessing.OneHotEncoder(categories=[genders, locations, browsers])
# Note that for there are missing categorical values for the 2nd and 3rd
# feature
X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses␣
 ↪Firefox']]
enc.fit(X)
enc.transform([['female', 'from Asia', 'uses Chrome']]).toarray()
```

[187]:
```
array([[1., 0., 0., 1., 0., 0., 1., 0., 0., 0.]])
```

[188]:
```
enc = preprocessing.OneHotEncoder(handle_unknown='infrequent_if_exist')
X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses␣
 ↪Firefox']]
enc.fit(X)
enc.transform([['female', 'from Asia', 'uses Chrome']]).toarray()
```

[188]:
```
array([[1., 0., 0., 0., 0., 0.]])
```

It is also possible to encode each column into `n_categories - 1` columns instead of `n_categories` columns by using the drop parameter. This parameter allows the user to specify a category for each feature to be dropped. This is useful to avoid co-linearity in the input matrix in some classifiers. Such functionality is useful, for example, when using non-regularized regression (LinearRegression), since co-linearity would cause the covariance matrix to be non-invertible:

[189]:
```
X = [['male', 'from US', 'uses Safari'],
     ['female', 'from Europe', 'uses Firefox']]
drop_enc = preprocessing.OneHotEncoder(drop='first').fit(X)
display(drop_enc.categories_)
display(drop_enc.transform(X).toarray())
```

```
[array(['female', 'male'], dtype=object),
 array(['from Europe', 'from US'], dtype=object),
 array(['uses Firefox', 'uses Safari'], dtype=object)]
```

```
array([[1., 1., 1.],
       [0., 0., 0.]])
```

[190]:
```
X = [['male', 'US', 'Safari'],
     ['female', 'Europe', 'Firefox'],
     ['female', 'Asia', 'Chrome']]
drop_enc = preprocessing.OneHotEncoder(drop='if_binary').fit(X)
display(drop_enc.categories_)
display(drop_enc.transform(X).toarray())
```

```
[array(['female', 'male'], dtype=object),
 array(['Asia', 'Europe', 'US'], dtype=object),
 array(['Chrome', 'Firefox', 'Safari'], dtype=object)]
```

```
array([[1., 0., 0., 1., 0., 0., 1.],
       [0., 0., 1., 0., 0., 1., 0.],
       [0., 1., 0., 0., 1., 0., 0.]])
```

[191]:
```
import pandas as pd

display(pd.DataFrame(
    drop_enc.transform(X).toarray(),
    columns=drop_enc.get_feature_names_out(
        ['gender', 'location', 'browser']
    )
))
```

```
   gender_male  location_Asia  location_Europe  location_US  browser_Chrome  \
0          1.0            0.0              0.0          1.0             0.0
1          0.0            0.0              1.0          0.0             0.0
2          0.0            1.0              0.0          0.0             1.0

   browser_Firefox  browser_Safari
0              0.0             1.0
```

```
1               1.0             0.0
2               0.0             0.0
```

1.7 Understanding the One-Hot Encoded Array

Original data:

Row	Gender	Location	Browser
1	male	US	Safari
2	female	Europe	Firefox
3	female	Asia	Chrome

The encoder is fitted using:

```
drop_enc = preprocessing.OneHotEncoder(drop='if_binary').fit(X)
```

1.7.1 What each column represents

`OneHotEncoder` creates indicator columns:

- **1** means the observation belongs to that category.
- **0** means it does not.

Categories are sorted alphabetically within each original feature. The output columns are:

Column position	Category
1	male
2	Asia
3	Europe
4	US
5	Chrome
6	Firefox
7	Safari

1.7.2 Why there is no female column

`drop='if_binary'` removes the first category from features with exactly two categories.

Gender has two categories: `female` and `male`. The encoder drops `female`, leaving only the `male` column:

- `male = 1` represents male.
- `male = 0` represents female in this dataset.

Location and browser each have three categories, so all their columns are retained.

1.7.3 Reading the output

```
array([[1., 0., 0., 1., 0., 0., 1.],
       [0., 0., 1., 0., 0., 1., 0.],
       [0., 1., 0., 0., 1., 0., 0.]])
```

With column labels:

Row	male	Asia	Europe	US	Chrome	Firefox	Safari
1	1	0	0	1	0	0	1
2	0	0	1	0	0	1	0
3	0	1	0	0	1	0	0

Therefore:

- **Row 1:** male, from US, uses Safari.
- **Row 2:** female, from Europe, uses Firefox.
- **Row 3:** female, from Asia, uses Chrome.

1.7.4 One-hot encoding versus ordinal encoding

Ordinal encoding assigns a category number. For location:

- Asia → 0
- Europe → 1
- US → 2

One-hot encoding creates a separate column for each category:

- Asia → `[1, 0, 0]`
- Europe → `[0, 1, 0]`
- US → `[0, 0, 1]`

US has category index 2, so one-hot encoding places a **1 at index 2** (the third position). It does not use 2 as the encoded value.

In the transformed X, the first column is the encoding of the feature with categories "male"/"female", while the remaining 6 columns are the encoding of the 2 features with respectively 3 categories each.

When handle_unknown='ignore' and drop is not None, unknown categories will be encoded as all zeros:

[192]:
```
drop_enc = preprocessing.OneHotEncoder(drop='first',
                                       handle_unknown='ignore').fit(X)
X_test = [['unknown', 'America', 'IE']]
drop_enc.transform(X_test).toarray()
```

```
/Users/alvaro/PycharmProjects/IATD-Machine-Learning-
Algorithms/.venv/lib/python3.12/site-
packages/sklearn/preprocessing/_encoders.py:242: UserWarning: Found unknown
categories in columns [0, 1, 2] during transform. These unknown categories will
```

```
be encoded as all zeros
  warnings.warn(
```

[192]:
```
array([[0., 0., 0., 0., 0.]])
```

[193]:
```
drop_enc = preprocessing.OneHotEncoder(drop='if_binary', sparse_output=False,
                                       handle_unknown='ignore').fit(X)
X_test = [['unknown', 'America', 'IE']]
X_trans = drop_enc.transform(X_test)
display(X_trans)
display(drop_enc.inverse_transform(X_trans))
```

```
/Users/alvaro/PycharmProjects/IATD-Machine-Learning-
Algorithms/.venv/lib/python3.12/site-
packages/sklearn/preprocessing/_encoders.py:242: UserWarning: Found unknown
categories in columns [0, 1, 2] during transform. These unknown categories will
be encoded as all zeros
  warnings.warn(
```

```
array([[0., 0., 0., 0., 0., 0., 0.]])
```

```
array([['female', None, None]], dtype=object)
```

1.8 8.3.4.1. Infrequent categories

OneHotEncoder and OrdinalEncoder support aggregating infrequent categories into a single output for each feature. The parameters to enable the gathering of infrequent categories are min_frequency and max_categories.

1. min_frequency is either an integer greater or equal to 1, or a float in the interval (0.0, 1.0). If min_frequency is an integer, categories with a cardinality smaller than min_frequency will be considered infrequent. If min_frequency is a float, categories with a cardinality smaller than this fraction of the total number of samples will be considered infrequent. The default value is 1, which means every category is encoded separately.
2. max_categories is either None or any integer greater than 1. This parameter sets an upper limit to the number of output features for each input feature. max_categories includes the feature that combines infrequent categories.

In the following example with OrdinalEncoder, the categories 'dog' and 'snake' are considered infrequent:

[194]:
```
X = np.array([['dog'] * 5 + ['cat'] * 20 + ['rabbit'] * 10 +
              ['snake'] * 3], dtype=object).T
display(X)
enc = preprocessing.OrdinalEncoder(min_frequency=6).fit(X)
display(enc.infrequent_categories_)
display(enc.transform(np.array([['dog'], ['cat'], ['rabbit'], ['snake']])))
```

```
array([['dog'],
       ['dog'],
       ['dog'],
```

```
       ['dog'],
       ['dog'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['cat'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['rabbit'],
       ['snake'],
       ['snake'],
       ['snake']], dtype=object)
```

```
[array(['dog', 'snake'], dtype=object)]
```

```
array([[2.],
       [0.],
       [1.],
       [2.]])
```

[195]:
```
import pandas as pd

categories = enc.categories_[0]
infrequent = enc.infrequent_categories_[0]
```

```
display(pd.DataFrame({
    "Category": categories,
    "Encoded value": enc.transform(categories.reshape(-1, 1)).ravel(),
    "Explanation": [
        "Infrequent: fewer than 6 occurrences; shares code 2"
        if category in infrequent
        else "Frequent: has its own code"
        for category in categories
    ]
}))
```

	Category	Encoded value	Explanation
0	cat	0.0	Frequent: has its own code
1	dog	2.0	Infrequent: fewer than 6 occurrences; shares c…
2	rabbit	1.0	Frequent: has its own code
3	snake	2.0	Infrequent: fewer than 6 occurrences; shares c…

[196]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
from sklearn import preprocessing

X = np.array([
    ['dog'] * 5 + ['cat'] * 20 + ['rabbit'] * 10 + ['snake'] * 3
], dtype=object).T

enc = preprocessing.OrdinalEncoder(min_frequency=6).fit(X)

# Explicitly include all four animals
animals = np.array(['dog', 'cat', 'rabbit', 'snake'])
codes = enc.transform(animals.reshape(-1, 1)).ravel().astype(int)
counts = [(X[:, 0] == animal).sum() for animal in animals]
infrequent = enc.infrequent_categories_[0]
is_infrequent = [animal in infrequent for animal in animals]

summary = pd.DataFrame({
    "Animal": animals,
    "Training count": counts,
    "Code": codes,
    "Explanation": [
        "Infrequent: shares code 2"
        if rare else "Frequent: has its own code"
        for rare in is_infrequent
    ]
})
```

```
display(summary)

# Plot the assigned codes
colours = [
    "orange" if rare else "steelblue"
    for rare in is_infrequent
]

fig, ax = plt.subplots(figsize=(10, 4))

ax.barh(animals, codes, color=colours)
ax.scatter(codes, animals, color=colours, s=70, zorder=3)

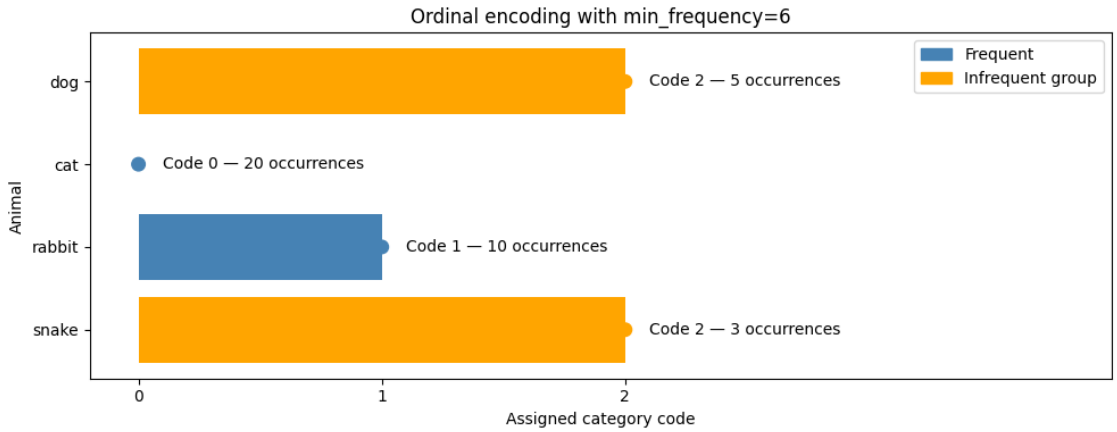
for i, (code, count) in enumerate(zip(codes, counts)):
    ax.text(
        code + 0.1, i,
        f"Code {code} - {count} occurrences",
        va="center"
    )

ax.invert_yaxis()
ax.set(
    xlabel="Assigned category code",
    ylabel="Animal",
    title="Ordinal encoding with min_frequency=6",
    xticks=[0, 1, 2],
    xlim=(-0.2, 4)
)

ax.legend(handles=[
    Patch(color="steelblue", label="Frequent"),
    Patch(color="orange", label="Infrequent group")
])

plt.tight_layout()
plt.show()
```

```
   Animal  Training count  Code                Explanation
0     dog               5     2  Infrequent: shares code 2
1     cat              20     0 Frequent: has its own code
2  rabbit              10     1 Frequent: has its own code
3   snake               3     2  Infrequent: shares code 2
```



OrdinalEncoder's **max_categories do not take into account missing or unknown categories**. Setting unknown_value or encoded_missing_value to an integer will increase the number of unique integer codes by one each. This can result in up to max_categories + 2 integer codes. In the following example, "a" and "d" are considered infrequent and grouped together into a single category, "b" and "c" are their own categories, unknown values are encoded as 3 and missing values are encoded as 4.

[197]:
```
X_train = np.array(
    [["a"] * 5 + ["b"] * 20 + ["c"] * 10 + ["d"] * 3 + [np.nan]],
    dtype=object).T
enc = preprocessing.OrdinalEncoder(
    handle_unknown="use_encoded_value", unknown_value=3,
    max_categories=3, encoded_missing_value=4)
_ = enc.fit(X_train)
X_test = np.array([["a"], ["b"], ["c"], ["d"], ["e"], [np.nan]], dtype=object)
enc.transform(X_test)
```

[197]:
```
array([[2.],
       [0.],
       [1.],
       [2.],
       [3.],
       [4.]])
```

[198]:
```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Patch

codes = enc.transform(X_test).ravel().astype(int)

labels = [
    "Missing (NaN)" if pd.isna(value) else str(value)
```

```
    for value in X_test[:, 0]
]

explanations = [
    "Infrequent: grouped with d",
    "Frequent category",
    "Frequent category",
    "Infrequent: grouped with a",
    "Unknown: not in training data",
    "Missing value"
]

colours = [
    "orange", "steelblue", "steelblue",
    "orange", "mediumpurple", "grey"
]

# Display all test categories and their meanings
display(pd.DataFrame({
    "Category": labels,
    "Code": codes,
    "Explanation": explanations
}))

# Plot the mapping
fig, ax = plt.subplots(figsize=(10, 5))

ax.barh(labels, codes, color=colours)
ax.scatter(codes, labels, color=colours, s=70, zorder=3)

for i, (code, explanation) in enumerate(zip(codes, explanations)):
    ax.text(
        code + 0.1, i,
        f"{code} - {explanation}",
        va="center"
    )

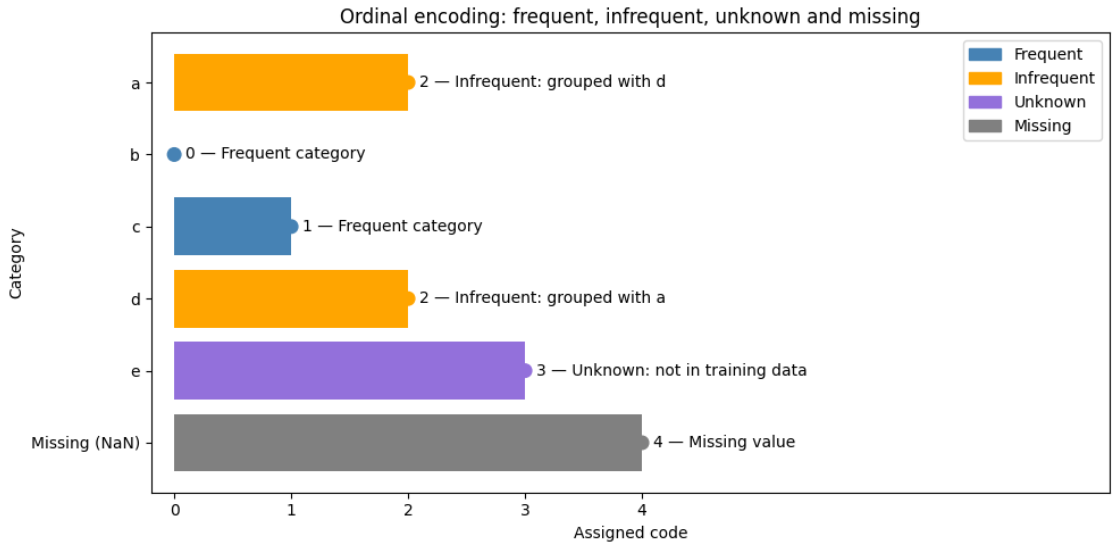
ax.invert_yaxis()
ax.set(
    xlabel="Assigned code",
    ylabel="Category",
    title="Ordinal encoding: frequent, infrequent, unknown and missing",
    xticks=range(5),
    xlim=(-0.2, 8)
)

ax.legend(handles=[
```

```
    Patch(color="steelblue", label="Frequent"),
    Patch(color="orange", label="Infrequent"),
    Patch(color="mediumpurple", label="Unknown"),
    Patch(color="grey", label="Missing")
])

plt.tight_layout()
plt.show()
```

	Category	Code	Explanation
0	a	2	Infrequent: grouped with d
1	b	0	Frequent category
2	c	1	Frequent category
3	d	2	Infrequent: grouped with a
4	e	3	Unknown: not in training data
5	Missing (NaN)	4	Missing value



Similarly, OneHotEncoder can be configured to group together infrequent categories:

[199]:
```
enc = preprocessing.OneHotEncoder(min_frequency=6, sparse_output=False).fit(X)
display(enc.infrequent_categories_)
display(enc.transform(np.array([['dog'], ['cat'], ['rabbit'], ['snake']])))
```

```
[array(['dog', 'snake'], dtype=object)]
```

```
array([[0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

[200]:
```
import pandas as pd
import dataframe_image as dfi
from pathlib import Path
from IPython.display import Image, display

animals = ['dog', 'cat', 'rabbit', 'snake']
X_test = np.array(animals, dtype=object).reshape(-1, 1)

encoded = enc.transform(X_test)

result = pd.DataFrame(
    encoded,
    index=pd.Index(animals, name="Animal"),
    columns=enc.get_feature_names_out(['animal'])
)

'''
display(
    result.style
        .format("{:.0f}")
        .map(lambda value:
             "background-color: lightblue; font-weight: bold"
             if value == 1 else "")
)
'''
# To make it compatible with nbconvert
styled_result = (
    result.style
    .format("{:.0f}")
    .map(
        lambda value:
        "background-color: lightblue; font-weight: bold"
        if value == 1 else ""
    )
)

image_path = Path("images/animal_encoding.png")
image_path.parent.mkdir(parents=True, exist_ok=True)

dfi.export(
    styled_result,
    str(image_path),
    table_conversion="chrome",
    dpi=200
)

display(Image(filename=str(image_path)))
```

Animal	animal_cat	animal_rabbit	animal_infrequent_sklearn
dog	0	0	1
cat	1	0	0
rabbit	0	1	0
snake	0	0	1

[201]:
```
X_test = np.array(
    [['dog'], ['cat'], ['rabbit'], ['snake']],
    dtype=object
)

encoded = enc.transform(X_test)

print("Encoder:", type(enc).__name__)
print("Shape:", encoded.shape)
print("Columns:", enc.get_feature_names_out(['animal']))

result = pd.DataFrame(
    encoded,
    index=pd.Index(['dog', 'cat', 'rabbit', 'snake'], name="Animal"),
    columns=enc.get_feature_names_out(['animal'])
)

# Print every column as plain text
print(result.to_string())
```

```
Encoder: OneHotEncoder
Shape: (4, 3)
Columns: ['animal_cat' 'animal_rabbit' 'animal_infrequent_sklearn']
        animal_cat  animal_rabbit  animal_infrequent_sklearn
Animal
dog            0.0            0.0                        1.0
cat            1.0            0.0                        0.0
rabbit         0.0            1.0                        0.0
snake          0.0            0.0                        1.0
```

By setting handle_unknown to ‘infrequent_if_exist’, unknown categories will be considered infrequent:

[202]:
```
enc = preprocessing.OneHotEncoder(
   handle_unknown='infrequent_if_exist', sparse_output=False, min_frequency=6)
enc = enc.fit(X)
enc.transform(np.array([['dragon']]))
```

[202]:
```
array([[0., 0., 1.]])
```

[203]:
```
enc.get_feature_names_out()
```

[203]:
```
array(['x0_cat', 'x0_rabbit', 'x0_infrequent_sklearn'], dtype=object)
```

Infrequent categories can also be configured using max_categories. In the following example, we set max_categories=2 to limit the number of features in the output. This will result in all but the 'cat' category to be considered infrequent, leading to two features, one for 'cat' and one for infrequent categories - which are all the others:

[204]:
```
enc = preprocessing.OneHotEncoder(max_categories=2, sparse_output=False)
enc = enc.fit(X)
display(enc.infrequent_categories_)
display(enc.transform([['dog'], ['cat'], ['rabbit'], ['snake']]))
```

```
[array(['dog', 'rabbit', 'snake'], dtype=object)]
```

```
array([[0., 1.],
       [1., 0.],
       [0., 1.],
       [0., 1.]])
```

[205]:
```
enc = preprocessing.OneHotEncoder(min_frequency=4, max_categories=3,␣
 ↪sparse_output=False)
enc = enc.fit(X)
display(enc.infrequent_categories_)
display(enc.transform([['dog'], ['cat'], ['rabbit'], ['snake']]))
```

```
[array(['dog', 'snake'], dtype=object)]
```

```
array([[0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

If there are infrequent categories with the same cardinality at the cutoff of max_categories, then the first max_categories are taken based on lexicon ordering. In the following example, "b", "c", and "d", have the same cardinality and with max_categories=2, "b" and "c" are infrequent because they have a higher lexicon order.

[206]:
```
X = np.asarray([["a"] * 20 + ["b"] * 10 + ["c"] * 10 + ["d"] * 10],␣
 ↪dtype=object).T
enc = preprocessing.OneHotEncoder(max_categories=3).fit(X)
enc.infrequent_categories_
```

[206]:
```
[array(['b', 'c'], dtype=object)]
```

The quoted explanation is inconsistent: the code uses `max_categories=3`, not 2, and b and c come **before** d in lexicographic order, not after it. The displayed output matches the code.

1.9 8.3.5. Discretization

[207]:
```
X = np.array([[ -3., 5., 15 ],
              [  0., 6., 14 ],
              [  6., 3., 11 ]])
est = preprocessing.KBinsDiscretizer(n_bins=[3, 2, 2], encode='ordinal').fit(X)

est.transform(X)
```

[207]:
```
array([[0., 1., 1.],
       [1., 1., 1.],
       [2., 0., 0.]])
```

1.10 Discretization: bins and value assignments

`KBinsDiscretizer` assigns each value to a bin, working independently on each feature (column).

With `n_bins=[3, 2, 2]`, feature 1 has three bins, and features 2 and 3 each have two bins. Bin numbers start at **0**.

1.10.1 Bin intervals

Feature	Bin 0	Bin 1	Bin 2
Feature 1	`[-∞, -1)`	`[-1, 2)`	`[2, +∞)`
Feature 2	`[-∞, 5)`	`[5, +∞)`	—
Feature 3	`[-∞, 14)`	`[14, +∞)`	—

A square bracket means the boundary is **included**; a round bracket means it is **excluded**.

For example, `[-1, 2)` includes −1 but excludes 2. Therefore, −1 belongs to bin 1, while 2 belongs to bin 2.

1.10.2 How each value is assigned

Row	Feature	Original value	Matching interval	Assigned bin
1	Feature 1	−3	`[-∞, -1)`	0
1	Feature 2	5	`[5, +∞)`	1
1	Feature 3	15	`[14, +∞)`	1
2	Feature 1	0	`[-1, 2)`	1
2	Feature 2	6	`[5, +∞)`	1
2	Feature 3	14	`[14, +∞)`	1
3	Feature 1	6	`[2, +∞)`	2

Row	Feature	Original value	Matching interval	Assigned bin
3	Feature 2	3	[-∞, 5)	0
3	Feature 3	11	[-∞, 14)	0

1.10.3 Final transformation

Row	Original values	Encoded bins
1	[-3, 5, 15]	[0, 1, 1]
2	[0, 6, 14]	[1, 1, 1]
3	[6, 3, 11]	[2, 0, 0]

Because `encode='ordinal'`, each original value is replaced by its **bin number**.

Be aware that one can specify custom bins by passing a callable defining the discretization strategy to FunctionTransformer. For instance, we can use the Pandas function pandas.cut:

[208]:
```
import pandas as pd
import numpy as np
from sklearn import preprocessing
bins = [0, 1, 13, 20, 60, np.inf]
labels = ['infant', 'kid', 'teen', 'adult', 'senior citizen']
transformer = preprocessing.FunctionTransformer(
    pd.cut, kw_args={'bins': bins, 'labels': labels, 'retbins': False}
)
X = np.array([0.2, 2, 15, 25, 97])
transformer.fit_transform(X)
```

[208]:
```
['infant', 'kid', 'teen', 'adult', 'senior citizen']
Categories (5, object): ['infant' < 'kid' < 'teen' < 'adult' < 'senior citizen']
```

[209]:
```
X = np.array([19,25,45])
transformer.fit_transform(X)
```

[209]:
```
['teen', 'adult', 'adult']
Categories (5, object): ['infant' < 'kid' < 'teen' < 'adult' < 'senior citizen']
```

1.11 8.3.5.2. Feature binarization

[210]:
```
X = [[ 1., -1.,  2.],
     [ 2.,  0.,  0.],
     [ 0.,  1., -1.]]

binarizer = preprocessing.Binarizer().fit(X)  # fit does nothing
display(binarizer)

print("Threshold:", binarizer.threshold)
```

```
display(binarizer.transform(X))
```

```
Binarizer()
```

```
Threshold: 0.0
```

```
array([[1., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.]])
```

It is possible to adjust the threshold of the binarizer:

[211]:
```
binarizer = preprocessing.Binarizer(threshold=1.1)
binarizer.transform(X)
```

[211]:
```
array([[0., 0., 1.],
       [1., 0., 0.],
       [0., 0., 0.]])
```

1.12 8.3.7. Generating polynomial features

[212]:
```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
X = np.arange(6).reshape(3, 2)
display(X)
poly = PolynomialFeatures(2)
display(poly.fit_transform(X))
```

```
array([[0, 1],
       [2, 3],
       [4, 5]])
```

```
array([[ 1.,  0.,  1.,  0.,  0.,  1.],
       [ 1.,  2.,  3.,  4.,  6.,  9.],
       [ 1.,  4.,  5., 16., 20., 25.]])
```

In some cases, only interaction terms among features are required, and it can be gotten with the setting interaction_only=True:

[213]:
```
X = np.arange(9).reshape(3, 3)
display(X)
poly = PolynomialFeatures(degree=3, interaction_only=True)
display(poly.fit_transform(X))
```

```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

```
array([[  1.,   0.,   1.,   2.,   0.,   0.,   2.,   0.],
       [  1.,   3.,   4.,   5.,  12.,  15.,  20.,  60.],
       [  1.,   6.,   7.,   8.,  42.,  48.,  56., 336.]])
```

1.13 8.3.7.2. Spline transformer

[214]:
```
import numpy as np
from sklearn.preprocessing import SplineTransformer
X = np.arange(5).reshape(5, 1)
display(X)
spline = SplineTransformer(degree=2, n_knots=3)
display(spline.fit_transform(X))
```

```
array([[0],
       [1],
       [2],
       [3],
       [4]])
```

```
array([[0.5  , 0.5  , 0.   , 0.   ],
       [0.125, 0.75 , 0.125, 0.   ],
       [0.   , 0.5  , 0.5  , 0.   ],
       [0.   , 0.125, 0.75 , 0.125],
       [0.   , 0.   , 0.5  , 0.5  ]])
```

[215]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import SplineTransformer

X = np.arange(5).reshape(5, 1)

spline = SplineTransformer(degree=2, n_knots=3)
X_spline = spline.fit_transform(X)

# Show how each input becomes four features
display(pd.DataFrame(
    X_spline,
    index=pd.Index(X.ravel(), name="Original x"),
    columns=["Basis 1", "Basis 2", "Basis 3", "Basis 4"]
))

# Use many intermediate x values to draw smooth curves
X_smooth = np.linspace(0, 4, 300).reshape(-1, 1)
Y_smooth = spline.transform(X_smooth)

fig, ax = plt.subplots(figsize=(9, 5))

for i in range(X_spline.shape[1]):
    line, = ax.plot(
        X_smooth.ravel(),
        Y_smooth[:, i],
```

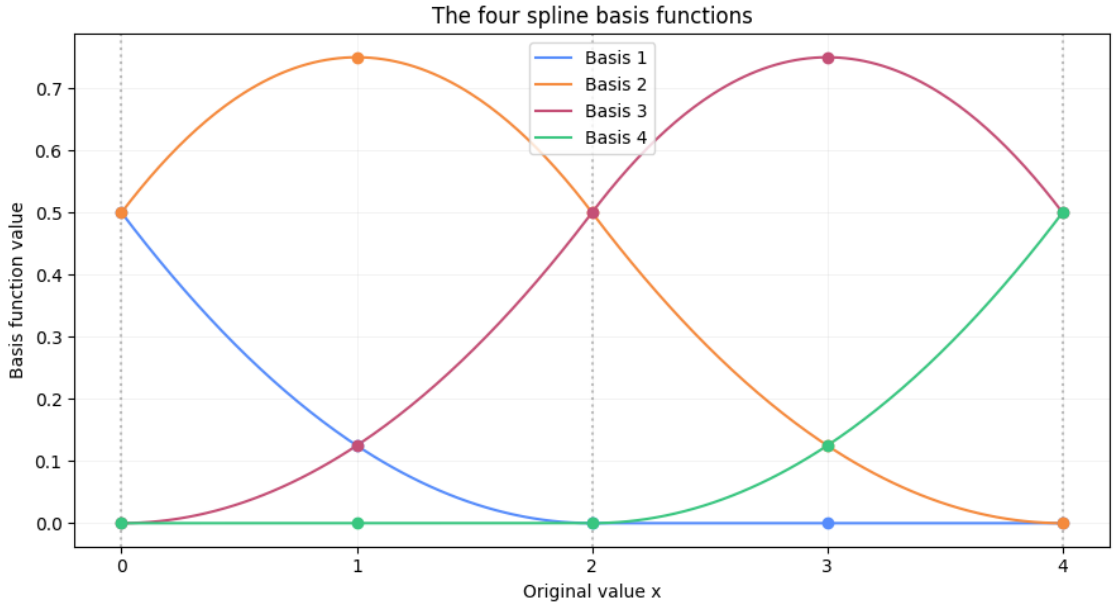
```
        label=f"Basis {i + 1}"
    )
    ax.scatter(
        X.ravel(),
        X_spline[:, i],
        color=line.get_color(),
        zorder=3
    )

# Mark the knots: 0, 2, 4
for knot in [0, 2, 4]:
    ax.axvline(knot, color="grey", linestyle=":", alpha=0.5)

ax.set(
    xlabel="Original value x",
    ylabel="Basis function value",
    title="The four spline basis functions",
    xticks=[0, 1, 2, 3, 4]
)
ax.legend()
ax.grid(alpha=0.2)

plt.tight_layout()
plt.show()
```

Original x	Basis 1	Basis 2	Basis 3	Basis 4
0	0.500	0.500	0.000	0.000
1	0.125	0.750	0.125	0.000
2	0.000	0.500	0.500	0.000
3	0.000	0.125	0.750	0.125
4	0.000	0.000	0.500	0.500



1.14 Understanding `SplineTransformer`

`SplineTransformer` converts each input value into several new features using smooth functions called **B-spline basis functions**.

These features allow a model, such as linear regression, to learn curved relationships.

1.14.1 Original data

The input contains one feature with five observations:

Observation	Original value (x)
1	0
2	1
3	2
4	3
5	4

1.14.2 Transformer settings

```
spline = SplineTransformer(degree=2, n_knots=3)
```

- **`degree=2`** means the basis functions consist of quadratic pieces joined smoothly.
- **`n_knots=3`** places three knots across the input range. In this example, they are at **0, 2, and 4**.
- Knots are locations where the polynomial pieces meet.

With the default settings, the number of output features is:

$$n_{\text{features}} = n_{\text{knots}} + \text{degree} - 1$$

Therefore:

$$3 + 2 - 1 = 4$$

The original feature becomes **four spline features**.

1.14.3 Transformed data

Original x	Basis 1	Basis 2	Basis 3	Basis 4
0	0.500	0.500	0.000	0.000
1	0.125	0.750	0.125	0.000
2	0.000	0.500	0.500	0.000
3	0.000	0.125	0.750	0.125
4	0.000	0.000	0.500	0.500

Each column corresponds to a different basis function. Each row contains the values of those four functions at a particular x.

For example, at $\mathbf{x = 1}$:

$$[B_1(1), B_2(1), B_3(1), B_4(1)] = [0.125,\ 0.750,\ 0.125,\ 0]$$

These are function values, not category codes or probabilities. For this example, the four values in each row sum to **1**.

1.14.4 How to read the plot

- The **horizontal axis** shows the original input value, x.
- The **vertical axis** shows the value of each basis function.
- Each coloured curve represents one output column.
- The dots show the transformed values for the original observations.
- The vertical dotted lines mark the knots at **0, 2, and 4**.

Choose an x value and read the height of each coloured curve. Those four heights form the corresponding row of the transformed data.

1.14.5 How this creates a fitted curve

`SplineTransformer` creates the basis features. A regression model then learns coefficients to combine them:

$$\hat{y}(x) = b + w_1 B_1(x) + w_2 B_2(x) + w_3 B_3(x) + w_4 B_4(x)$$

Here:

- $B_1(x), \ldots, B_4(x)$ are the spline basis functions.
- $w_1, \ldots, w_4$ are weights learned from training data.
- b is the intercept.

The prediction can curve as x changes because the basis functions are curved, even though the model combines them using a weighted sum.

1.15 8.3.8. Custom transformers

Often, you will want to convert an existing Python function into a transformer to assist in data cleaning or processing. You can implement a transformer from an arbitrary function with FunctionTransformer. For example, to build a transformer that applies a log transformation in a pipeline, do:

[216]:
```
import numpy as np
from sklearn.preprocessing import FunctionTransformer
transformer = FunctionTransformer(np.log1p, validate=True)
X = np.array([[0, 1], [2, 3]])
display(X)
# Since FunctionTransformer is no-op during fit, we can call transform directly
display(transformer.transform(X))
```

```
array([[0, 1],
       [2, 3]])
```

```
array([[0.        , 0.69314718],
       [1.09861229, 1.38629436]])
```

numpy.log1p Return the natural logarithm of one plus the input array, element-wise.

[217]:
```
for i in range(4):
    print(f"For value {i} the natural logarithm ln(1+i) is: {np.log1p(i)}")
```

```
For value 0 the natural logarithm ln(1+i) is: 0.0
For value 1 the natural logarithm ln(1+i) is: 0.6931471805599453
For value 2 the natural logarithm ln(1+i) is: 1.0986122886681098
For value 3 the natural logarithm ln(1+i) is: 1.3862943611198906
```

[218]:
```
import warnings
warnings.filterwarnings("error", message=".*check_inverse*.",
                        category=UserWarning, append=False)
```